A Primer on MeTTa Pure/Core

The Semantic Waist, Declaration-Aware Families, and the Proof/Execution Split

AI-assisted conceptual note for continuity across development sessions

2026-03-13

Abstract

This note consolidates the main architectural and mathematical ideas that emerged while refining the MeTTa Pure/Core development. Its purpose is not to record commit history. Instead, it isolates the durable conceptual picture that should continue to guide future work.

The central thesis is that the project should converge to a *small trusted semantic waist* on the proof side, with a separate runtime execution seam on the operational side, both fed from a shared elaborated core. The proof-side waist should remain generic and theoremic; ordinary families, recursors, and structural recursion should arise from declaration-aware machinery above that waist rather than from family-specific syntax hacks. Prototype evaluators and surface conveniences may exist, but they must be downstream of the trusted kernel rather than silently shaping it.

The note explains the clean architecture, the main lessons learned from drift and recovery, the declaration-aware route to ordinary families, the correct shape of generated dependent recursors, the role of positivity and substitution, the separation between δ-unfolding and ι-reduction, and the current status of the connection to MeTTaIL and the MM2/MORK runtime seam. It is intended to serve as a didactic primer for future AI-assisted development sessions.

Contents

1 Purpose and Scope **1**

2 The Architectural Thesis **2**

3 The Semantic Waist **2**
- 3.1 What the waist is . . . 2
- 3.2 Why the waist must stay small . . . 3
- 3.3 What the waist is not . . . 3

4 The Proof-Side Reference Must Be Theoremic, Not Evaluator-Driven **3**

5 Legacy Ambient Presentations versus the Trusted Kernel **4**

6 Declaration-Aware Extensions above the Waist **4**
- 6.1 Why declarations are the right extension point . . . 4
- 6.2 The current layering idea . . . 5
- 6.3 Why prefix discipline matters . . . 5
- 6.4 What still needs to become first-class . . . 5

7 Ordinary Families Must Be Declarations, Not Kernel Hacks **6**
7.1 The main principle . . . 6
7.2 What a genuine family declaration must contain . . . 6
7.3 Dependent telescopes are mandatory . . . 6

8 Dependent Recursors Are the Canonical Target **7**
8.1 The canonical shape . . . 7
8.2 The Nat example . . . 7
8.3 The List example . . . 7

9 Positivity and Indexed Families **7**

10 Substitution and Generated Computation **8**
10.1 Parallel substitution is the right semantic primitive . . . 8
10.2 The correct design rule . . . 8

11 δ-Unfolding versus ι-Reduction **8**
11.1 What should stay in a declaration specification . . . 8
11.2 What should live above it . . . 9

12 The Runtime Frontier and the Meaning of "a Strain of MeTTa" **9**
12.1 The honest current status . . . 9

13 Lessons from Drift and Recovery **9**
13.1 Progress can be fake if the trust boundary moves . . . 10
13.2 The right response to a bad shortcut is usually retreat, not polish . . . 10
13.3 A good proof failure is valuable . . . 10

14 Development Discipline for Future AI-Assisted Sessions **10**
14.1 Core rules . . . 10
14.2 Questions every future change should answer . . . 11

15 A Robustness Checklist **11**

16 A Suggested Near-Term Roadmap **11**

17 Glossary of Key Terms **12**

18 Conclusion **13**

1 Purpose and Scope

The goal of this note is to make the Pure/Core project legible in a way that is stable across individual coding sessions. The emphasis is on *conceptual architecture*, not on temporary implementation detail.

The note has four jobs:

1. identify the mathematically clean center of the project;
2. explain which extensions belong *above* that center rather than *inside* it;

3. record the most important failure modes that must not be repeated;
4. provide a compact primer that future development sessions can use as a normative reference.

The intended reader is someone who may be new to the repository but needs to understand the design constraints immediately and accurately.

2 The Architectural Thesis

The working architectural thesis can be stated in one sentence:

> Surface MeTTa should elaborate to a shared middle layer, and that shared middle layer should support two distinct but connected targets: a proof-oriented semantic target and a runtime execution target.

The three main layers are therefore:

1. **Surface MeTTa.** The user-facing language, notation, parser, syntax sugar, and future ergonomic layer.
2. **Elaborated Core.** A shared, explicit intermediate layer used to mediate between syntax and semantics.
3. **Two honest targets.**
   a. a proof-side target, centered on a small trusted dependent kernel;
   b. an execution-side target, centered on a real operational seam such as the MM2/MORK runtime stack.

This architecture matters because it prevents two opposite confusions:

1. *runtime reduction is not the proof kernel*;
2. *proof semantics and runtime execution are not permanently unrelated silos either*.

The shared elaborated core is the place where those worlds can meet honestly.

3 The Semantic Waist

3.1 What the waist is

A *semantic waist* is the smallest trusted layer that is still rich enough to support the key theoremic services of the proof side. In the present design, that waist is a small dependent kernel with:

- universes (currently the tiny `Type0`/`Type1` story),
- dependent function types (Π),
- dependent pair types (Σ),
- identity types,
- ordinary variables and binders,

- global constants,
- a small, explicit reduction relation,
- typing, confluence, subject reduction, and definitional equality.

The crucial point is that this waist should be *generic*. It should not be polluted by family-specific constructors such as hardcoded Nat, Bool, or recursor nodes if the intended final architecture is a general family mechanism.

3.2 Why the waist must stay small

A small waist has four advantages.

1. **Metatheory stays local.** Confluence, subject reduction, algorithmic typing, and conversion are harder to trust when the syntax grows opportunistically.
2. **Extensions stay honest.** New user-visible features must justify themselves as elaboration or declaration mechanisms above the waist rather than by mutating the kernel whenever a demo is desired.
3. **The trust boundary is explicit.** One can say exactly what is trusted and what is merely derived.
4. **Future automation becomes safer.** AI agents are much less likely to introduce architectural drift when the kernel surface is visibly narrow and well guarded.

3.3 What the waist is not

The semantic waist is *not*:

- a parser;
- a convenience evaluator;
- a proof explorer UI;
- a staging area for temporary built-ins;
- a promise that every future user-level construct already has kernel computation rules.

Those can all exist, but none of them should define the trusted core.

4 The Proof-Side Reference Must Be Theoremic, Not Evaluator-Driven

One of the most important lessons is that the proof-side *reference* should be the checked theoremic boundary, not a prototype evaluator.

The conceptual center of the proof side should be:

1. typed checking into the trusted kernel;
2. canonicalization by a theoremically sanctioned notion such as complete development (written here as cdev);

3. checked bridges between syntax, kernel terms, and shared artifacts.

A small evaluator can still be useful, but only if it is treated as a derived explorer. The key rule is:

> Prototype evaluation may *test* the kernel, but it may not *shape* the kernel.

This distinction matters because evaluator-driven development tends to create exactly the wrong incentives. It rewards short-term demonstrations and can make non-final shortcuts look like conceptual progress. The clean alternative is to let theoremic checking and canonicalization remain authoritative, while any executable evaluator is explicitly secondary.

5 Legacy Ambient Presentations versus the Trusted Kernel

It is perfectly reasonable for a project like this to contain more than one presentation of "Pure":

- a legacy or ambient presentation over a broader host language;
- a smaller, more explicit kernel-level presentation used as the trusted center.

The danger is not the existence of both. The danger is forgetting which one is trusted.
The decisive lesson from the recent confluence repair is the following:

> A theorem, its reduction relation, and its fragment assumptions must all live on exactly the same fragment.

If confluence is only proved for a fragment of terms, then the conversion relation used by downstream typing must be fragment-safe as well. One must not silently claim Church–Rosser for a broader ambient relation and then sneak in fragment facts only at the call site. The correct move is to tighten the relation itself or tighten the theorem boundary, not to fabricate missing hypotheses.

This is more than a technicality. It is a design law:

1. fragment membership should be an upstream concept,
2. binder and purity facts should be upstream utilities,
3. confluence should be downstream of those facts,
4. subject reduction should be downstream of confluence,
5. no proof should quietly range over a larger syntax than its lemmas can actually support.

6 Declaration-Aware Extensions above the Waist

6.1 Why declarations are the right extension point

Once the small kernel includes *global constants*, the clean next step is not to add more syntax constructors. It is to add a declaration-aware layer above the waist.

Conceptually, this means introducing:

- an environment of declarations,

- a declaration-aware typing judgment,
- a declaration-aware reduction relation,
- well-formedness conditions on the environment.

This allows one to treat families, constructors, and recursors as ordinary named declarations rather than new kernel AST nodes.

6.2 The current layering idea

A principled layering for declaration-aware work is:

1. **Minimal operational environment well-formedness.** Enough to justify lookup and basic δ-unfolding soundly.
2. **Ordered signatures.** Enough to talk about declaration order, shadowing discipline, and prefix reasoning.
3. **Prefix discipline.** Enough to say that declaration types and unfolding values only mention names from earlier entries.
4. **Full family-formation theory.** A later layer that justifies families, constructors, generated recursors, and equation schemas as a genuine semantic mechanism rather than a list of pilots.

This is a good architecture because it does not pretend the first operational invariant is already the whole declaration theory. The intermediate layers make order semantically meaningful before the full family machinery is present.

6.3 Why prefix discipline matters

A pure "global final environment" view is too weak. It can hide circularity, self-reference, and order-dependence. Prefix discipline repairs this by making left-to-right growth real:

- fresh names against the prefix,
- values typecheck against the prefix,
- declaration types mention only earlier names,
- declaration values mention only earlier names,
- no self-unfolding at the point of introduction.

This is not yet the final family theory. It is an honest bridge toward it.

6.4 What still needs to become first-class

The next theoremic bundling should introduce a stronger environment invariant, for example a conceptually named declaration-environment well-formedness record, rather than leaving individual obligations scattered.

Such a record should eventually bundle at least:

- well-formed declared types,

- well-typed declared values,
- closure/scoping requirements,
- a stratification or acyclicity discipline if unfolding becomes more central to conversion.

7 Ordinary Families Must Be Declarations, Not Kernel Hacks

7.1 The main principle

Ordinary inductive families such as Unit, Bool, Nat, and later indexed families such as Vec, should be *instances of a general family-declaration mechanism.* They should not be introduced by mutating the trusted syntax of the small kernel whenever a new example is desired.

This principle has a direct corollary:

> If a feature is meant to come from a later general mechanism, it should not be hardcoded now as a family-specific shortcut in the trusted lane.

7.2 What a genuine family declaration must contain

A real ordinary-family declaration layer should be able to express, at minimum:

- the family name,
- a *parameter telescope*,
- an *index telescope*,
- the resulting sort,
- constructors with full *typed telescopes*,
- a *generated recursor* specification,
- *equation schemas* for generated computation,
- *positivity/admissibility* evidence.

Several important negative conclusions follow immediately:

1. a list of closed types is not enough when dependency matters;
2. constructor arity is not constructor typing;
3. a closed left-hand side/right-hand side pair is not yet a generic equation schema under binders;
4. boolean flags are not acceptable substitutes for proof obligations in the trusted lane.

7.3 Dependent telescopes are mandatory

If telescopes are represented by closed terms only, then later binders cannot depend on earlier ones. That is already too weak for ordinary family work.

So the family record should use a genuinely binder-aware telescope rather than a flat list of closed types. The family constant type should then be derived from that telescope by iterated Π-formation, rather than stored redundantly and risking divergence between two representations.

8 Dependent Recursors Are the Canonical Target

8.1 The canonical shape

For a non-indexed family F with parameters Γ_p and constructors in order $c_1, \ldots, c_k$, the canonical generated recursor should be the dependent eliminator:

$$\Pi\Gamma_p.\Pi P : F\,\Gamma_p \to \mathsf{Type0}.\Pi case_1 : CaseTy_1.\cdots \Pi case_k : CaseTy_k.\Pi x : F\,\Gamma_p.P\,x.$$

For indexed families, the parameters still come *before* the motive, while the indices are bound *inside* the motive and major premise. In other words, the right rule is:

parameters outside the motive; indices inside the motive.

Case arms should appear in constructor declaration order. This keeps the relationship between declarations, generated types, and generated computation rules stable and reviewable.

8.2 The Nat example

The canonical dependent eliminator for naturals is:

$$\mathsf{Nat.rec} : \Pi P : \mathsf{Nat} \to \mathsf{Type0}.P\,(\mathsf{Nat.zero}) \to \big(\Pi n : \mathsf{Nat}.\ P\,n \to P\,(\mathsf{Nat.succ}\ n)\big) \to \Pi n : \mathsf{Nat}.\ P\,n.$$

A non-dependent programming recursor,

$$\Pi R : \mathsf{Type0}.\ R \to (\mathsf{Nat} \to R \to R) \to \mathsf{Nat} \to R,$$

may be a useful *derived interface*, but it should not be the primary generator target. The dependent eliminator is the canonical source of truth.

8.3 The List example

For a future list family $\mathsf{List} : \Pi A : \mathsf{Type0}.\ \mathsf{Type0}$, the canonical dependent eliminator is:

$$\mathsf{List.rec} : \Pi A : \mathsf{Type0}.\Pi P : \mathsf{List}\,A \to \mathsf{Type0}.P\,(\mathsf{List.nil}\ A) \to \big(\Pi x : A.\ \Pi xs : \mathsf{List}\,A.\ P\,xs \to P\,(\mathsf{List.cons}\ A\ x\ xs)\big) \to \Pi$$

This is the right conceptual template for future generated recursors.

9 Positivity and Indexed Families

Strict positivity should be checked over the *full constructor type*, not just over the parametric part. Once indices are present, recursive occurrences can hide in index-bearing targets and argument positions that a parameter-only check would miss.

However, positivity and target-shape checking should be separated into two orthogonal constraints:

1. **Uniform parameters.** The parameter prefix of every recursive family occurrence must be exactly the declared parameters, in the expected positions.

2. **Flexible indices.** Index arguments may be arbitrary well-scoped terms over the constructor context.

This matters for indexed families such as

$$\mathsf{Vec} : \Pi A : \mathsf{Type0}.\ \mathsf{Nat} \to \mathsf{Type0},$$

where a target like $\mathsf{Vec}\ A\ (\mathsf{Nat.succ}\ n)$ must be allowed.

A clean implementation strategy is therefore to decompose applications into a head and an argument spine, check the family head, split the arguments into parameter and index portions, enforce exact matching on the parameter prefix, and permit arbitrary scoped terms for the indices.

10 Substitution and Generated Computation

10.1 Parallel substitution is the right semantic primitive

A useful conceptual simplification is this: if the kernel already has parallel substitution as its basic operation, then generated recursor reduction does not require a new foundation. It requires a new *wrapper*.

In other words, a simultaneous substitution API is semantically sufficient. What is usually missing is only an ergonomic interface for substituting several constructor arguments and induction hypotheses into a case body in the right de Bruijn order.

A thin convenience operation such as `substVec` or `instMany` is a good idea. But it should be understood as a thin layer over the existing parallel substitution theory rather than as a new primitive.

10.2 The correct design rule

The important design rule is:

> Do not rebuild the semantics around repeated one-variable substitution if a clean simultaneous substitution is already available.

Repeated one-variable substitution is much easier to get wrong, especially under binders and when constructor arguments and induction hypotheses interleave.

11 δ-Unfolding versus ι-Reduction

A declaration-aware kernel needs to distinguish two different mechanisms:

1. **δ-unfolding.** A named constant unfolds to its stored value by declaration lookup.
2. **ι-reduction.** A generated recursor computes on a saturated application of the recursor to a constructor.

These are not the same thing, and they should not be modeled by the same data field.

11.1 What should stay in a declaration specification

A declaration specification should remain focused on the ordinary environment layer:

- name,
- type,
- optional value for ordinary δ-unfolding,
- well-formedness conditions relative to the current signature.

11.2 What should live above it

Generated recursor computation should live in a *separate* semantic layer, for example a generated-recursor rule bundle or family-level computation record. That higher layer should package:

- the generated recursor declaration,
- the generated ι-rules,
- typing proofs for those rules,
- coverage and arity discipline,
- whatever later confluence or conversion facts are needed.

This separation keeps the ordered-signature story clean. It also prevents constant lookup from being muddled with eliminator computation.

12 The Runtime Frontier and the Meaning of "a Strain of MeTTa"

The Pure/Core project is not aiming to replace the runtime semantics with proof semantics. The goal is subtler.

The proof-side target should remain a mathematically isolated dependent kernel. The execution-side target should remain a real operational seam with genuine runtime behavior, currently exemplified by the MM2/MORK line.

The interesting question is therefore not "is the proof kernel literally the runtime?" but rather:

> What is the cleanest elaborated-core fragment that can lower honestly to both sides?

This reframes the relation between Pure and execution in a productive way.

12.1 The honest current status

The current honest position is:

- the proof side already has a trusted checking and canonicalization center;
- the runtime side already has a real execution seam;
- there is not yet a theoremically established direct Pure-to-runtime execution bridge for the full proof fragment;
- the next mathematically meaningful connection is an *intentional dual-target fragment* of elaborated core.

This is a healthy status, not a deficiency. It keeps the architecture honest.

13 Lessons from Drift and Recovery

Several hard lessons emerged from the development process. They are worth recording explicitly.

13.1 Progress can be fake if the trust boundary moves

A project can appear to make fast progress while actually losing architectural integrity. Typical examples include:

- adding family-specific kernel constructors because a demonstration seems easier that way;
- promoting a convenience evaluator to reference status;
- widening a theorem statement beyond the fragment actually proved;
- leaving stale examples and docs in place after a semantic reset;
- putting pilot modules on the same import surface as the trusted core.

The appearance of momentum is not enough. The relevant question is always:

> Did the new capability arise from the intended final mechanism, or from a shortcut that would have to be deleted later?

13.2 The right response to a bad shortcut is usually retreat, not polish

When a trusted layer has absorbed a shortcut that is not part of the intended final design, the correct move is usually to back it out rather than to make it nicer. Cosmetic polish does not convert a non-final mechanism into a final one.

13.3 A good proof failure is valuable

The confluence break in the ambient legacy layer was a useful event because it revealed that the theorem boundary was overclaimed. Proof failures that expose mismatched abstraction boundaries are healthy. They should trigger conceptual repair, not theorem-patching theater.

14 Development Discipline for Future AI-Assisted Sessions

The project needs explicit design rules that future sessions can enforce.

14.1 Core rules

R1. No temporary trusted constructors. Never add a trusted kernel constructor, eliminator, typing rule, or reduction rule unless it belongs to the intended final architecture.

R2. No evaluator-driven semantics. Prototype evaluators may exist, but they may not determine the trusted kernel shape.

R3. No family-specific shortcuts when a general mechanism is the goal. If a feature is supposed to arise from a generic family declaration system, do not hardcode a family-specific version in the trusted lane.

R4. Match theorem and fragment. The syntax fragment, conversion relation, and theorem statement must range over the same object language.

R5. Keep trust labels explicit. Every new component should be classifiable as trusted kernel, declaration extension, pilot/specification, or experimental tooling.

R6. Synchronize code, docs, and examples. Any semantic change must update examples and documentation in the same pass.

R7. Prefer proof objects to booleans in the trusted lane. A boolean flag may be acceptable during exploration, but the trusted theory should expose propositions, structures, and theorems rather than hope encoded as data.

14.2 Questions every future change should answer

Before accepting a kernel-adjacent change, one should be able to answer:

1. Does this belong inside the small waist, or above it?
2. If it touches syntax, why can constants plus declarations not express it instead?
3. Is the new semantics theoremic, or is it being inferred from a tool?
4. Are the fragment assumptions of all downstream theorems still honest?
5. Does this make future generalization easier, or does it create more bespoke cases?

15 A Robustness Checklist

The Pure/Core project can be considered to have reached a robust kernel stage when the following become true.

1. **New families are declaration instances, not syntax changes.** Adding Nat, Bool, List, or an indexed family should no longer require edits to the trusted kernel AST.
2. **Recursors are generated, not hand-added.** The dependent recursor for a family should come from a generic generation procedure justified by the family declaration mechanism.
3. **The declaration preservation stack is complete.** In particular: one-step δ preservation, one-step declaration-aware preservation, and star preservation.
4. **The proof-side reference is stable.** The checked kernel plus canonicalization remains the reference; evaluators and UI layers remain downstream.
5. **The bridge to execution is intentional rather than accidental.** There is at least one explicitly identified elaborated-core fragment that lowers honestly to both the proof side and the runtime side.

16 A Suggested Near-Term Roadmap

The following sequence is conceptually conservative and mathematically aligned with the architecture.

1. **Keep the small kernel frozen.** No family-specific syntax growth.
2. **Make trust boundaries visible.** Separate tiny trusted-kernel umbrellas from declaration-extension umbrellas and pilot/experimental umbrellas.

3. **Bundle declaration-environment well-formedness more tightly.** Turn the current scattered obligations into an explicit semantic record.

4. **Complete the declaration preservation tower.** Prove one-step δ preservation, then general declaration-aware one-step preservation, then star preservation.

5. **Introduce a binder-aware generic family declaration record.** Use dependent telescopes, derived family types, typed constructors, generated dependent recursors, equation schemas, and proof-valued admissibility fields.

6. **Realize Unit as the first real instance.** It is the simplest family and the best proving ground for the generic record.

7. **Add generated ι semantics above the signature layer.** Keep ordinary declaration lookup and generated recursor computation distinct.

8. **Then add Bool, then Nat.** Each should become an instance of the same mechanism, not a special case.

9. **Only afterwards revisit stronger canonicalization and the proof/ runtime frontier.** By then, the new family layer will be stable enough to connect outward.

17 Glossary of Key Terms

Semantic waist
The smallest trusted proof-side layer rich enough to support checking, canonicalization, and later extensions.

Elaborated core
The explicit shared middle layer between surface MeTTa and the proof/runtime targets.

Canonicalization / cdev
The theoremically authoritative proof-side reduction or normalization anchor, distinct from any convenience evaluator.

δ-unfolding
Constant unfolding by declaration lookup.

ι-reduction
Generated eliminator computation on a recursor applied to a constructor.

Declaration-aware kernel
The small kernel together with declaration environments, declaration-aware typing, and declaration-aware reduction.

Prefix discipline
The requirement that declaration types and values may only mention earlier names, so signature order has semantic force.

Ordinary family
A strictly positive inductive family, possibly with parameters and later with indices.

Dependent recursor
: The canonical eliminator whose motive may depend on the major premise (and, for indexed families, on the indices).

Dual-target fragment
: A deliberately chosen fragment of elaborated core that lowers honestly to both the proof side and the runtime execution side.

18 Conclusion

The main conceptual picture is now clear.

The Pure/Core project should be organized around a small generic trusted kernel, a declaration-aware extension above that kernel, a future general family mechanism that treats families as declarations rather than syntax hacks, and a separate runtime execution seam connected through a shared elaborated core.

The most important design lesson is that correctness and architectural honesty must outrank short-term demonstrations. A project like this becomes durable only when each layer is justified by the intended final mechanism, not by temporary convenience.

If future development keeps the semantic waist small, keeps declarations ordered and meaningful, generates families and recursors from general records, and connects to execution through explicit dual-target fragments rather than wishful identification, then the project has a credible path toward a robust and mathematically well-isolated version of MeTTa Pure/Core.